

KOÇ UNIVERSITY

COMP 437/537 – Intelligent User Interfaces

ITIM: Intelligent Tutoring and Interface for Mathematics

IUI Final Report

Amr Zaki Salih | Yagiz Efe Acar

May 2026

Abstract

Mathematics education at every level suffers from a persistent structural challenge that is that students work through problems in isolation, receive delayed feedback, and frequently develop incorrect mental models such as misconceptions or "buggy rules", patterns representing a common student misconception that compound over time into deeper conceptual gaps. Traditional automated tutoring systems either rely on expensive human expertise or provide superficial feedback that ignores the process of problem solving by rewarding or penalising only the final answer.

ITIM, the Interactive Tutoring Interface for Mathematics is designed to bridge this gap. It provides a rich digital learning environment combining a natural multi-page digital handwriting canvas, a conversational AI tutor, real-time handwriting to LateX OCR, symbolic mathematics verification using SymPy, and a cognitive monitoring engine inspired by Anderson's ACT-R architecture. Students can write mathematics as they would on paper, receive instant context-aware feedback, and be monitored continuously for signs of frustration, misconceptions or cognitive overload with the system intervening proactively and adaptively when such signals are detected.

This report describes the full design and implementation of ITIM , covering its backend architecture, cognitive model, mathematics domain, exercise system, frontend, and data flows, as well as the design rationale and trade-offs behind the key technical decisions. The user study design and its results are presented in subsequent sections.

Contents

1 Introduction **4**

1.1 Motivation 4

1.2 Project Goals 5

1.3 Target Users 5

1.4 Contributions 5

2 Background and Related Work **6**

2.1 Intelligent Tutoring Systems 6

2.2 ACT-R and Knowledge Tracing 6

2.3 Conversational LLM Assistance 6

2.4 Handwriting Recognition for Mathematics 7

3 System Design **7**

3.1 High-Level Architecture 7

3.2 Backend Architecture 9

3.3 Cognitive Architecture 9

3.4 Mathematics Domain 10

3.4.1 Expression Verification 10

3.4.2 Buggy Rule Detection 10

3.4.3 Step Evaluation 11

3.5 Exercise System 11

3.6 Frontend and IUI Design 11

3.7 Complete Data Flows 12

4 User Study Design **15**

4.1 Hypotheses 15

4.2 Study Design 15

4.3 Participants 15

4.4 Procedure 16

4.5 Measures 16

5 Analysis and Results **16**

5.1 Objective Performance 16

5.2 NASA-TLX Results 17

5.3 ITIM-Specific Ratings 18

5.4 Qualitative Feedback 20

5.5 Discussion 20

6 Limitations and Future Work **21**

6.1 Limitations 21
6.2 Future Work 22
6.3 Position in the IUI Field 22

7 Conclusion **23**

A Appendix: Complete File Reference **24**

1. Introduction

Software and digital tools have transformed many areas of education, yet mathematics tutoring remains stubbornly difficult to automate effectively. A student working through a calculus problem alone has no way to know whether their approach is the right way until they reach the end by which time a misconception silently applied ten steps earlier has corrupted the entire solution.

Prior Intelligent Tutoring Systems called ITS have approached the problem in different ways. Rule-based systems such as the LISP Tutor [2] could model procedural knowledge accurately but required extensive manual authoring for every new domain. More recent systems such as ASSISTments [5] deliver adaptive problem sets at scale but still rely on structured multiple-choice or typed input, and their feedback is limited to right/wrong judgements on final answers.

The recent convergence of LLMs with strong mathematical reasoning, Vision APIs capable of reading handwritten notation, and computer algebra systems (CAS) for symbolic verification opens a design space that previous systems could not occupy. ITIM exploits this convergence to build a tutoring system in which a student can write mathematics by hand, have that writing automatically interpreted, receive pedagogically grounded feedback from a conversational AI tutor, and be supported proactively when the system detects that they are struggling all in the same seamless interface.

1.1. Motivation

Three observations motivate the design of ITIM.

At first the Naturalness of input matters. Indeed, requiring students to express mathematics in a formal syntax whether LateX, a CAS input language, or a multiple-choice grid imposes a cognitive overhead that detracts from the mathematical thinking itself. Students at the middle school and high school levels in particular are accustomed to working on paper; forcing them to translate between their natural notation and a system notation creates friction that can mask genuine understanding.

Secondly, the feedback timing is critical. Research on formative feedback consistently shows that immediate, specific feedback at the step level substantially outperforms delayed summary feedback for procedural learning [7]. Automated systems are uniquely positioned to provide this immediacy at scale. The challenge is to make the feedback pedagogically rich rather than merely correct/incorrect explaining why a step is wrong and what the student should consider next.

And at third, the cognitive state monitoring can prevent disengagement. Students who experience sustained frustration or cognitive overload tend to disengage, give up, or develop avoidance strategies rather than seeking help [3]. A system that can detect these states through observable

behavioral signals such as idle time, eraser frequency, and thus intervene supportively which can keep students productively engaged where a passive system would lose them.

1.2. Project Goals

ITIM was developed with five primary goals.

- **Naturalness of interaction.** Students should be able to write mathematics as they would on paper, without learning any formal syntax.
- **Adaptive feedback.** The system should adapt its level of support (scaffolding level) to each student's demonstrated competence and current cognitive state.
- **Misconception detection.** Commonly occurring algebraic errors should be detected symbolically and explained contextually.
- **Exercise continuity.** Exercises should be loadable from an imported document PDF, generatable from detected mathematical concepts, or creatable on demand.
- **Multi-modal interaction.** Students should be able to interact by typing, speaking, or writing on the canvas; the tutor should respond in text or synthesised speech.

1.3. Target Users

ITIM targets students at three educational levels. At first, University students working on calculus, linear algebra, and differential equations represent the most technically demanding use case. Secondly, High school students covering algebra, trigonometry, and analytic geometry form the primary target population. And finally, Middle school students working on arithmetic, basic algebra, and fractions represent the broadest accessibility target. The system dynamically adapts its language and pedagogical strategy based on the detected difficulty level of the current exercise and the student's performance history.

1.4. Contributions

This report makes the following contributions to the IUI field.

- We describe the design and full-stack implementation of ITIM, an intelligent tutoring system that integrates natural digital handwriting, real-time Vision-LLM OCR, symbolic mathematics verification, LLM-based conversational tutoring, and ACT-R-inspired cognitive monitoring into a single coherent educational experience.

- We present and justify the key design decisions and trade-offs in building a WebSocket-first, LLM-augmented tutoring system, providing practical guidance for future work in adaptive educational interfaces.
- We describe the complete data flow architecture that connects handwriting input to pedagogically appropriate feedback in near real-time, including the proactive intervention loop that distinguishes ITIM from purely reactive tutoring systems.

2. Background and Related Work

2.1. Intelligent Tutoring Systems

Intelligent Tutoring Systems have been studied since the 1970s. Early systems such as the LISP Tutor [2] demonstrated that individualised, step-level feedback can produce learning gains comparable to human tutoring, but required extensive manual knowledge engineering for every new domain. Systems such as ASSISTments [5] later scaled ITS to large student populations through web delivery and data-driven hint generation, but retained a structured input model that excludes natural mathematical writing. ITIM builds on this lineage while adding the natural handwriting input and mixed-initiative conversational interaction that prior systems lacked.

2.2. ACT-R and Knowledge Tracing

Anderson's Adaptive Control of Thought — Rational or ACT–R architecture [1] models human cognition as a set of declarative memory chunks whose activation level determines retrieval probability. The architecture has been highly influential in ITS design: Corbett and Anderson's Knowledge Component model [4], derived from ACT-R, became the basis for most modern knowledge tracing approaches. ITIM implements a simplified ACT-R engine that retains the educationally relevant aspects such as chunk activation, cognitive load assessment, and knowledge tracing without the parameter calibration overhead of a full implementation.

2.3. Conversational LLM Assistance

The application of LLMs to interactive assistance is closely related to the work of Ross et al. [6] on the Programmer's Assistant, a conversational programming aid grounded in the user's source code. Their study with 42 participants demonstrated that a conversational LLM interface provides value substantially beyond autocompletion: it enables emergent behaviours, context-aware follow-up questions, and learning effects. Participants were surprised by the quality of contextual responses and reported that the assistant helped them understand concepts, not only generate code. ITIM searches to apply the same design insight to mathematics education: the

conversational interface allows students to ask follow-up questions, contest feedback, request hints at varying levels of explicitness, and interact in their preferred natural language.

A key architectural difference is that Ross et al. deliberately chose a reactive design. It means that the assistant never takes conversational initiative to avoid disrupting the user's flow. ITIM takes a mixed-initiative approach: proactive interventions are suppressed when the student is actively writing (in-flow detection), but are triggered by sustained inactivity, repeated erasure, or cascading failures. This is motivated by the educational observation that students who are genuinely stuck benefit from a timely prompt, whereas students who are actively working benefit from uninterrupted concentration.

2.4. Handwriting Recognition for Mathematics

Digital ink recognition for mathematical notation has been an active research area for two decades. Early systems relied on hand-crafted grammars and symbol classifiers; more recent approaches use sequence-to-sequence and attention models. ITIM takes a pragmatic approach, indeed, rather than training a specialised recognition model, it leverages GPT-4o's Vision API, which demonstrates strong performance on handwritten mathematical content out of the box. The trade-off consisting of a higher per-call latency and API cost versus zero training overhead is explicitly acknowledged, and fine-tuning a smaller vision model is identified as a future work item.

3. System Design

3.1. High-Level Architecture

At the highest level, ITIM is a full-stack web application composed of four major layers. A *React 19 frontend* delivers the interactive canvas and chat UI to any modern browser or tablet. A *Python/FastAPI backend* exposes a persistent WebSocket endpoint for all real-time communication and a REST API for non-real-time operations. A set of *domain modules* handles mathematics, cognitive modelling, and exercise management entirely on the server side. External AI services such as OpenAI GPT-4o or Groq as a low-latency alternative provide language understanding, Vision-based OCR, and speech synthesis.

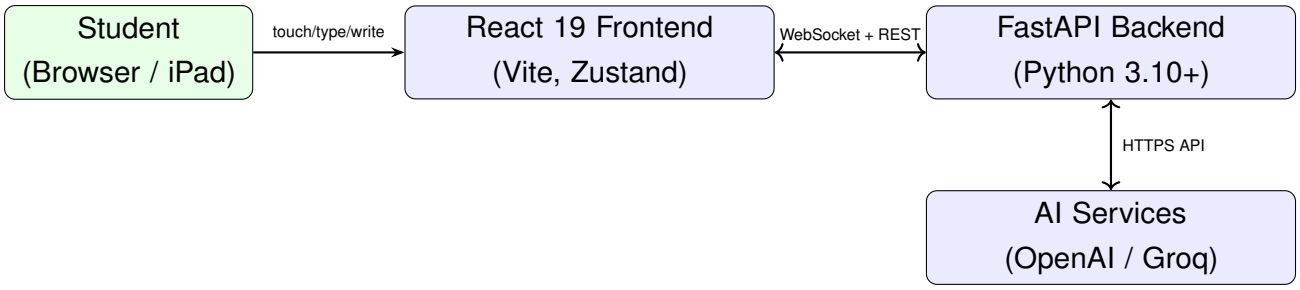


Figure 1: High-level system topology of ITIM. The student interacts with a React frontend that communicates in real time with a FastAPI backend; the backend delegates language understanding, OCR, and speech to external AI services.

The complete technology stack is presented in Table 1. The choice of Python with FastAPI and React 19 reflects the need for high-concurrency async I/O on the backend — handling concurrent WebSocket sessions and non-blocking LLM calls — combined with a reactive, component-based UI. SymPy provides a rigorous CAS for answer verification that is independent of the LLM, ensuring that mathematical correctness judgements are not subject to hallucination.

Table 1: Core technology stack of ITIM v2.

Layer	**Technology**	**Role**
Backend runtime	Python 3.10+	Application language
Web framework	FastAPI + Uvicorn	REST and WebSocket server
Math engine	SymPy	Symbolic algebra, step generation, verification
LaTeX parsing	latex2sympy2	Convert LaTeX to SymPy ASTs
Image processing	Pillow (PIL)	Canvas rendering for OCR upload
LLM / Vision	OpenAI GPT-4o	Chat, OCR, document extraction, explanation
LLM alternative	Groq (Llama / Orpheus)	Low-latency fallback LLM
Speech input	Whisper (OpenAI / Groq)	Automatic speech recognition
Speech output	OpenAI TTS / Groq Orpheus	Text-to-speech synthesis
Database	SQLite	Exercise library, student profiles
Frontend	React 19 + Vite + Zustand	UI components and state management
Math rendering	KaTeX (CDN)	Render LaTeX in the browser
PDF rendering	PDF.js	Client-side PDF page rasterisation

3.2. Backend Architecture

All real-time interactions such as strokes, tutor messages, OCR results and step feedback flow through a persistent WebSocket connection rather than HTTP polling. This is a deliberate architectural choice: stroke feedback must be delivered in tens of milliseconds, and server-push proactive interventions are impossible without a persistent channel. The trade-off is that WebSocket connections are stateful; horizontal scaling beyond a single instance would require a shared session store such as Redis.

The WebSocket handler acts as a central dispatcher: each incoming JSON message carries a `type` field used to route it to the appropriate coroutine. The two most important pipelines are the stroke pipeline and the user message pipeline.

When a stroke arrives, the backend records it, updates the ACT-R engine's behavioural state, and schedules a debounced OCR task: if no new stroke arrives within 0.8 seconds, the canvas is rendered to PNG and sent to the Vision API. This debounce window is a practical compromise between responsiveness and API cost. An additional check-in fires after 20 seconds of inactivity to catch students who have stopped writing without explicitly submitting an answer.

When a student message arrives, the backend first tries to classify it if it is a contest detection, an explicit answer extraction or an exercise control keywords such as "skip", "give up", "next step" before falling back to a general LLM call through the Tutor orchestrator. This layered routing means that common structured interactions are handled cheaply and deterministically, while open-ended conversation is handled by the LLM.

Connection state is encapsulated in a `SessionState` object instantiated once per WebSocket connection. This scoping means multiple concurrent students share no mutable state, making the system horizontally safe at the session level.

3.3. Cognitive Architecture

The system's most distinctive IUI contribution is its proactive cognitive monitoring. ITIM implements a simplified model inspired by Anderson's ACT-R architecture [1], tracking student knowledge through *chunks* which are discrete units of mathematical skill, each with an activation level in $[0, 1]$. Chunk activation is boosted on success and decayed on failure, converging over time to a stable estimate of the student's mastery for each skill.

Every 15 seconds, the intervention loop assesses cognitive load by combining five behavioural signals. It asses the idle time since the last stroke or message, eraser count in the current window, recent incorrect submissions, writing flow which is defined as four or more strokes within ten seconds, and average chunk activation for the current exercise. The result is one of three load levels:

- The first one is **low** corresponding to the student making steady progress meaning that no intervention is needed.
- The second one is **medium** corresponding to some difficulty or inactivity detected; a subtle hint may be appropriate after a grace period.
- The third one is **High** meaning that multiple simultaneous signals (prolonged idle, repeated erasing, cascading failures) had immediate supportive intervention triggered.

Critically, interventions are suppressed when the student is in flow which means that they are in pause when the users are actively and continuously writing. This directly addresses the tension identified by Ross et al. [6] in the Programmer's Assistant study, where participants worried that a proactive assistant would disrupt their concentration. ITIM resolves this by treating active writing as a do-not-interrupt signal.

When an intervention is warranted, the `PedagogicalAgent` selects an appropriate response through a four-level scaffolding strategy. Indeed, it goes from a vague directional nudge at level 1, through a guided question and partial explanation at levels 2 and 3, to a fully explicit step-by-step hint at level 4. The scaffolding level is maintained per student and updated automatically: success decrements it, failure increments it. The LLM then generates the actual message text using a prompt that includes the current exercise, the content visible on the student's board, the detected cognitive state, and a strict instruction to respond in the student's own language.

3.4. Mathematics Domain

3.4.1. Expression Verification

A central challenge in automated mathematics tutoring is verifying whether a student's answer is correct without rejecting valid alternative forms. ITIM addresses this through a cascade of four strategies applied in order: structural AST matching (fast, handles commutativity), SymPy symbolic equivalence (handles algebraic identities such as $\sin^2 x + \cos^2 x = 1$), equation side comparison (for expressions containing =), and numeric spot-checking (three random substitutions as a last resort for cases where symbolic simplification fails). Stopping at the first match avoids unnecessary computation while maximising coverage.

3.4.2. Buggy Rule Detection

Beyond binary correct or incorrect judgements, ITIM detects *buggy rules* which are named patterns corresponding to common algebraic misconceptions such as the distributivity error for example ($a(b + c) \rightarrow ab + c$) or the sign error ($-(a + b) \rightarrow -a + b$). Each rule is matched symbolically using SymPy; when a match is found, the LLM generates a personalised contextual

explanation rather than a generic error message. The system also runs a `BuggyRuleLearner`: errors that match no known rule are accumulated, and after three similar occurrences a new rule is proposed to the catalogue, making the misconception detector self-improving over time.

3.4.3. Step Evaluation

For structured exercises, the `StepTracker` manages a three-state machine which are Idle, Active and Completed and evaluates partial work by sending the full visible board content to the LLM alongside the problem statement and expected answer. Pure SymPy verification is used for final answers; LLM evaluation is reserved for intermediate steps, where symbolic equivalence alone cannot assess whether the student's reasoning is sound. This hybrid approach balances reliability with pedagogical richness.

3.5. Exercise System

Exercises in ITIM can originate from three sources: imported documents (PDF or image), on-demand generation, or the built-in SymPy fallback generators. When the imported document is itself an exercise sheet, the system behaves differently: rather than generating new exercises from detected concepts, the Vision API reads the document, recognises and extracts the existing questions directly, and proposes them to the student as there are. In both cases, the backend sends an `exercisesproposed` message containing the full set of available exercises; the chat panel renders them as clickable cards, and the student can begin whichever exercise they choose in any order. For documents that are not exercise sheets such as a course textbook or lecture notes for example, the pipeline falls back to the generative approach which means that a lightweight keyword heuristic identifies mathematical concepts without an LLM call, and the LLM generates three difficulty-levelled exercises per concept, each verified by SymPy before entering the SQLite library. This dual behaviour ensures that the exercise system adapts naturally to the type of document the student imports, whether they are working directly from a problem set or studying from reference material.

3.6. Frontend and IUI Design

The frontend's most distinctive IUI feature is its three-layer canvas corresponding to (`Page.jsx`) which a read-only background layer for imported PDF pages, a drawing layer for handwritten strokes, and a floating layer for typed text zones. The canvas uses the Pointer Events API rather than separate mouse and touch handlers, providing unified stylus, touch, and mouse input with palm rejection by making touch events wider than 30 px as discarded and pointer capture for out-of-bounds tracking. Strokes are rendered incrementally per pointer-move event for minimal perceived latency.

The chat panel supports two interaction modes. Standard text chat provides a conversational interface with inline KaTeX-rendered mathematics and exercise proposal cards. The Hands-free conversation mode as for him designed for tablet users holding a stylus, uses browser-side Voice Activity Detection to detect speech onset and silence, automatically transcribing utterances via Whisper and playing back the tutor's reply via TTS. This mode removes the need to switch between writing and typing, preserving the student's physical engagement with the canvas.

The LateX Confirm Toast appears after each OCR cycle and shows the student a KaTeX-rendered preview of what the system read from their handwriting. The student can dismiss, correct, or submit it. This design makes the OCR process transparent and gives the student agency over what the system acts on, avoiding silent misreadings.

A final IUI consideration is the language transparency. In fact, every LLM call carries a system instruction to respond in the same language as the student's last message. The system therefore works in English and French, and other languages, without any language-selection UI element, reflecting a broader principle of reducing interface friction wherever possible.

3.7. Complete Data Flows

Figures 2, 3, and 4 summarise the three main data flows: handwriting to feedback, conversational tutoring, and proactive intervention.

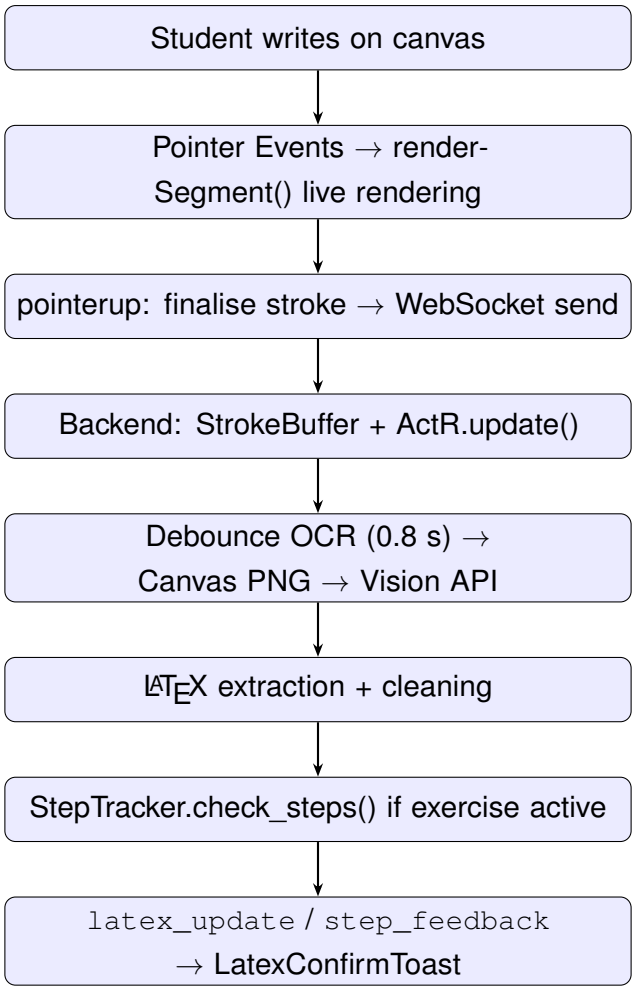


Figure 2: Handwriting-to-feedback data flow.

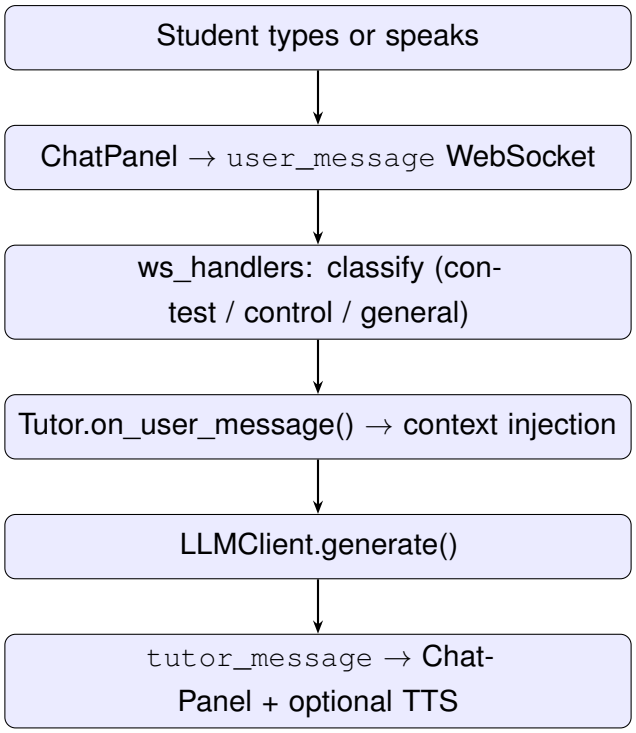


Figure 3: Conversational tutoring data flow.

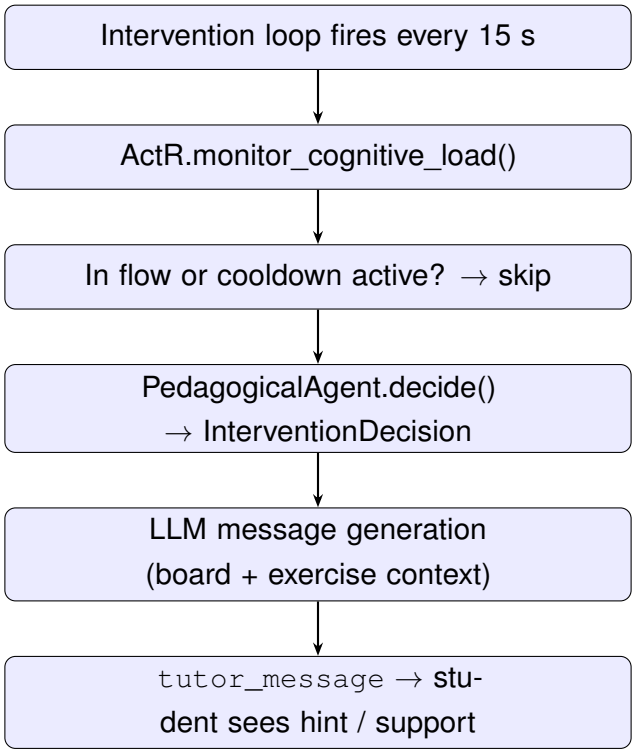


Figure 4: Proactive cognitive intervention flow.

4. User Study Design

4.1. Hypotheses

The central question of our user study was whether ITIM improves both the *objective outcome* and the *subjective experience* of solving mathematics exercises compared to working with pen and paper alone. We tested three hypotheses.

The first hypothese is that students using ITIM will complete a significantly higher number of questions correctly than students solving the same exercises without any digital assistance.

The second is that students using ITIM will report lower perceived effort and mental demand on the NASA Task Load Index (NASA-TLX) than when working without assistance.

The third is that students will report higher satisfaction with their own performance when using ITIM than when working without assistance.

4.2. Study Design

We adopted a **counterbalanced within-subjects design** to control for subject difficulty and individual ability differences. Two mathematics exercise of end of highschool / first year of university level sheets of comparable difficulty, Subject A and Subject B were prepared. Fourteen participants were divided into two groups of seven.

- **Group 1** solved Subject A *without* ITIM (pen and paper only), then solved Subject B *with* ITIM.
- **Group 2** solved Subject A *with* ITIM, then solved Subject B *without* ITIM.

This counterbalancing ensures that any observed difference between conditions cannot be attributed to one exercise sheet being intrinsically easier than the other, nor to a systematic learning effect benefiting only one group. Each participant therefore contributed one observation in the *with ITIM* condition and one in the *without ITIM* condition, making the design fully paired.

4.3. Participants

Fourteen participants were recruited from the university student population. All were enrolled in mathematics-related courses and had prior experience with pen-and-paper problem solving. None had used ITIM before the study. Participants were assigned to groups to achieve balanced representation across mathematical backgrounds.

4.4. Procedure

Each session began with a brief introduction to the study protocol. In the ITIM condition, participants received a five-minute orientation to the system covering the handwriting canvas, the chat panel, and how to submit steps. They were encouraged to use all available features which are the handwriting, the chat, and the voice but were not required to follow any particular interaction pattern. In the paper condition, participants used only pen and paper with no digital assistance. Both conditions were time-limited and conducted under the same environmental conditions.

4.5. Measures

We collected the following measures after each condition.

- **Objective performance.** The number of questions answered correctly out of the total on each sheet, graded against a solution key.
- **NASA Task Load Index (NASA-TLX).** A widely used subjective workload scale administered after each condition, covering five dimensions on a 0–100 scale: mental demand, physical demand, temporal demand, effort, and frustration. We also included a sixth item measuring perceived performance corresponding to a self-rated satisfaction with one's results, where higher scores indicate greater satisfaction.
- **ITIM-specific ratings** which collected in the ITIM condition only. Three additional items rated 0–100: OCR accuracy ("the system correctly recognised my math strokes"), AI tutor relevance ("the chat interventions were helpful when I was stuck"), and cognitive support ("the system detected my misconceptions").
- **Qualitative feedback.** Open-ended questions asking participants to identify the most innovative or helpful feature and to report any issues or suggestions.

5. Analysis and Results

5.1. Objective Performance

Table 2 reports the mean number of questions answered correctly per condition. The pattern is consistent across both groups: participants completed substantially more questions correctly when using ITIM than when working on paper alone.

Table 2: Mean number of questions answered correctly by condition (out of 5).

Condition	Group	Mean	SD
Subject A, without ITIM	Group 1	2.71	1.38
Subject B, with ITIM	Group 1	4.14	0.69
Subject A, with ITIM	Group 2	4.00	0.82
Subject B, without ITIM	Group 2	2.29	1.60
All with ITIM	Both	**4.07**	0.75
All without ITIM	Both	**2.50**	1.46

Pooling across both groups, students answered a mean of 4.07 questions correctly with ITIM versus 2.50 without, a difference of 1.57 questions per sheet. A Wilcoxon signed-rank test on the paired within-subject differences confirmed this effect was statistically significant ($W = 4.5$, $p = .010$). H1 is thus supported.

5.2. NASA-TLX Results

Table 3 summarises the NASA-TLX scores across all conditions. Lower scores are better for mental demand, physical demand, effort, and frustration; higher scores are better for perceived performance.

Table 3: NASA-TLX scores by condition (0–100 scale; mean $\pm$ SD). For demand, effort, and frustration, lower is better. For perceived performance, higher is better.

Dimension	With ITIM	Without ITIM	Mean diff.	p-value
Mental demand	40.4 ± 20.3	48.2 ± 19.2	-7.9	.134
Physical demand	37.5 ± 27.3	40.7 ± 26.0	-3.2	.416
Effort	36.4 ± 21.1	48.5 ± 22.4	-13.8	**.044**
Frustration	30.7 ± 22.9	29.6 ± 22.0	$+1.1$	.906
Perceived perf.	77.1 ± 12.7	59.3 ± 23.7	$+17.9$	**.023**

Effort was significantly lower with ITIM ($W = 13.5$, $p = .044$), with participants reporting a mean reduction of 13.8 points. This result supports H2 for the effort dimension. Mental demand and physical demand showed trends in the expected direction (reductions of 7.9 and 3.2 points respectively) but did not reach significance at $\alpha = .05$, likely due to the small sample size.

Frustration showed no meaningful difference between conditions ($p = .906$), suggesting that ITIM neither relieved nor increased emotional tension.

Perceived performance was significantly higher with ITIM ($W = 16.5$, $p = .023$), with a mean increase of 17.9 points. Students felt considerably more satisfied with their own results when working with the system. H3 is thus supported.

5.3. ITIM-Specific Ratings

Table 4 shows the ratings for the three ITIM-specific items collected in the system condition.

Table 4: ITIM-specific feature ratings collected in the system condition ($n = 14$; 0–100 scale).

Item	**Mean**	**SD**	**Median**
OCR accuracy	81.1	15.7	90.0
AI tutor relevance	73.2	25.3	80.0
Cognitive support	67.9	28.9	75.0

OCR accuracy received the highest rating (mean 81.1, median 90), indicating that the handwriting recognition was reliable for most participants most of the time. AI tutor relevance (mean 73.2) and cognitive support (mean 67.9) were also rated positively, confirming that participants found the conversational tutor helpful and that the system's misconception detection was perceived as meaningful.

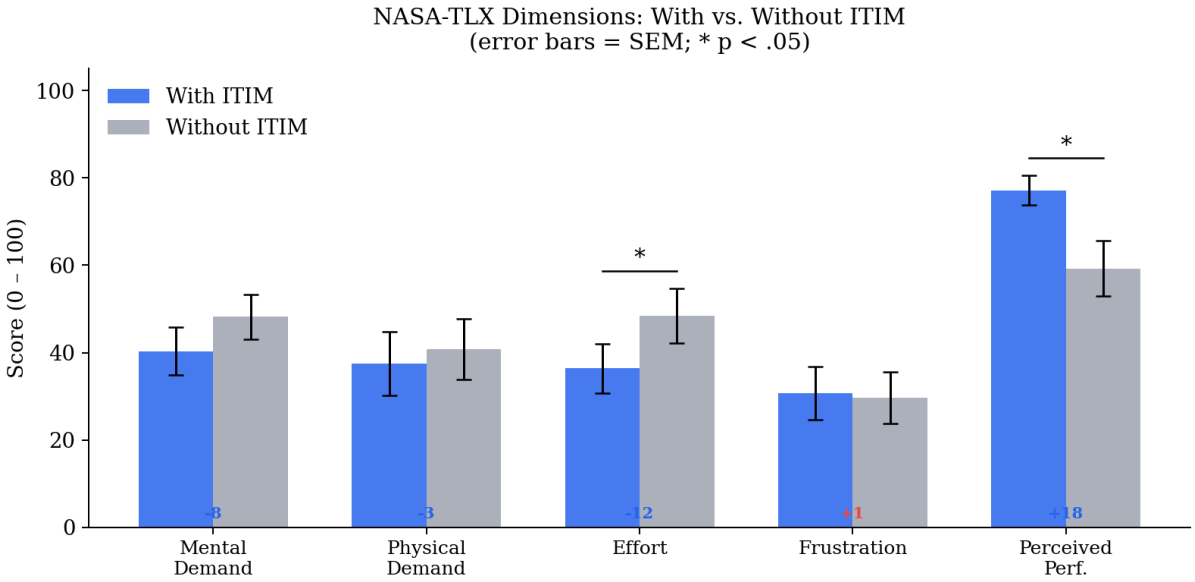


Figure 5: NASA-TLX scores by dimension for the with-ITIM and without-ITIM conditions ($n = 14$; error bars = SEM). Significant differences are marked with * ($p < .05$). The signed difference (with minus without) is shown at the base of each pair: blue indicates ITIM is favourable, red indicates the reverse.

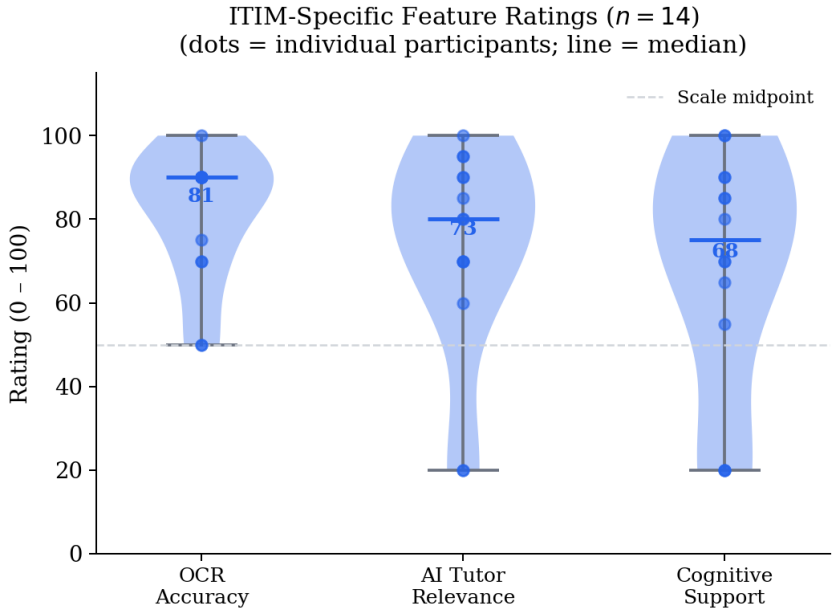


Figure 6: Distribution of ITIM-specific feature ratings ($n = 14$; 0–100 scale). Dots show individual participant scores; the horizontal line marks the median; the shaded violin shows the full distribution. The dashed line marks the scale midpoint (50). Bold numbers report the group mean.

5.4. Qualitative Feedback

Open-ended responses revealed a consistent set of themes. The most frequently praised features were the detection and recognition of handwriting ("Detection of what is written", "Extracting handwriting"), the step-by-step exercise structure ("Step-by-step segmentation of question", "Iterative solving instead of writing the answer"), and the AI companion aspect ("Nice to have an AI companion which helps you solve the question. It can be helpful for Math related courses. I use ChatGPT for this kind of classes", "Getting help when stuck is beneficial").

The most commonly reported issues were latency ("Delays", "Time delays", "Latency"), **OCR errors** ("Recognition error — found the correct answer but did not detect it", "Answered some questions correctly and AI told me it was wrong"), and **AI repetition or miscommunication** ("Repetition of the AI, making mistakes", "Doesn't always understand that I found the answer and wants me to do all the steps", "Consideration of another function while I was trying to solve another problem, so a miscommunication happened").One participant expressed disappointment with the system's pedagogical behaviour, noting that it "didn't guide me to solve a problem, nor step-by-step, nor explanation of a concept". This suggests that for some users the conversational tutor did not meet the expectation of structured, Socratic guidance which is a finding that points directly to the need for a more reliable scaffolding trigger and clearer onboarding about how to request step-by-step help from the system.

5.5. Discussion

The results largely aligned with our expectations. The strongest and most consistent effect was on objective performance. Indeed, students completed significantly more questions correctly with ITIM, with a mean improvement of 1.57 questions per sheet and low variance in the ITIM condition (SD = 0.75 versus 1.46 without), suggesting that ITIM not only raised average performance but made it more consistent across participants.

The effort reduction is also coherent with the system's design. By providing step-level feedback and proactive hints, ITIM reduces the cognitive work of checking one's own reasoning and recovering from dead ends. The absence of a significant effect on mental demand is less surprising on reflection: working with ITIM requires students to interpret feedback, interact with the chat panel, and manage the confirmation toasts, which may offset some of the demand reduction from not having to verify steps independently.

The frustration result is the most nuanced. The near-zero difference ($+1.1$ points, $p = .906$) suggests that ITIM neither relieved frustration nor added to it. Qualitative feedback points to two competing effects: some participants found the system reassuring and supportive, while others were frustrated by latency, OCR misrecognitions, and the system insisting on completing all steps when they felt they had already found the answer. These opposing forces appear to cancel

out at the group level. Reducing frustration caused by system errors is therefore an important target for future development.

6. Limitations and Future Work

6.1. Limitations

At first, the sample size was a limitation. Indeed, with 14 participants, the study is underpowered for strong statistical conclusions. Several NASA-TLX dimensions (mental demand, physical demand) showed trends in the expected direction that did not reach significance, likely due to insufficient power. A larger study would be needed to detect these smaller effects reliably.

Secondly, the most frequently cited complaint in qualitative feedback was response latency. Each LLM call for step evaluation adds approximately 1 to 3 seconds of delay, and OCR debouncing adds a further 0.8 seconds after the student stops writing. For students working quickly, this latency may break the natural flow of solving a problem. Reducing it, through for example, a fine-tuned smaller vision model, local SymPy pre-checking, or streaming partial responses is the highest-priority technical improvement.

Third, OCR errors and false negatives were a limitation. Several participants reported that the system failed to recognise a correct answer they had written, leading to incorrect negative feedback. This is a trust-critical failure; indeed, being told a correct answer is wrong is more damaging than receiving no feedback at all. The current pipeline relies entirely on GPT-4o Vision; fine-tuning a smaller model on mathematical handwriting data would likely improve accuracy and reduce this class of error.

The simplified ACT-R is limiting. The cognitive model omits procedural memory, spreading activation, and the full declarative memory retrieval equations of Anderson's original theory. The simplified version captures the most educationally relevant aspects such as knowledge tracing and cognitive load but may be less accurate for predicting long-term learning outcomes.

Also, the study is based on a single-session design. The study measured short-term performance and immediate subjective experience. It does not address whether ITIM produces durable learning gains or whether students become dependent on the system's scaffolding over time. A longitudinal study would be needed to answer these questions.

Moreover, although Subject A and Subject B were designed to be of comparable difficulty, residual differences in difficulty or topic familiarity could confound the comparison. Counterbalancing mitigates but does not eliminate this threat.

6.2. Future Work

Several directions emerge naturally from the study findings and the system's current limitations. The most urgent is reducing latency: fine-tuning a compact vision model on mathematical handwriting data would cut OCR response time and API costs, while streaming partial LLM responses to the frontend as they are generated would further reduce the perceived wait time that participants cited most frequently. A closely related improvement concerns step evaluation. Several participants reported that the system required them to complete all intermediate steps even after they had already found the correct final answer. Adding a mechanism to detect a correct answer at any point in the step sequence — and acknowledging it immediately rather than continuing to prompt for further steps — would remove one of the most cited sources of friction, and would make the system feel more like a responsive partner than a rigid procedure. On the system architecture side, collaborative mode is already partially built: the stroke relay between sessions is implemented, and the remaining work is frontend merge logic and backend session linking. Enabling full multi-student real-time collaboration would open ITIM v2 to group study sessions and classroom use, extending the co-creative interaction model that Ross et al. [6] identified as valuable in the software engineering domain to mathematics education. A complementary addition would be a teacher-facing dashboard showing per-student knowledge traces, common misconceptions across a class, and exercise completion rates. The underlying data is already tracked in the SQLite profile store; the remaining work is a visualisation layer. From a research perspective, the study's single-session design is its most significant limitation, and a longitudinal evaluation spanning several weeks with the same participants would allow us to address questions that the current data cannot: whether ITIM v2 produces durable learning gains, whether students become dependent on scaffolding over time, and how accurately the ACT-R model's knowledge traces converge toward true mastery estimates. Finally, incorporating spreading activation and procedural memory into the cognitive model would bring it closer to Anderson's full ACT-R theory and improve the precision of intervention timing, while extending the mathematics domain to geometric constructions, function plotting, and proof-based reasoning would substantially broaden the range of courses for which the system is applicable.

6.3. Position in the IUI Field

ITIM sits at the intersection of three active research areas in the IUI community which are intelligent tutoring systems, mixed-initiative conversational agents, and natural input for technical domains. Its contribution assistance space is the addition of proactive cognitive monitoring and the integration of a domain-specific verification layer (SymPy) that grounds the system's correctness judgements independently of the LLM. This hybrid approach using LLM for naturalness and explanation, CAS for correctness, ACT-R for timing is a design pattern that may generalise to other technical education domains beyond mathematics.

7. Conclusion

ITIM is a comprehensive intelligent tutoring system for mathematics education that integrates several technically demanding components into a coherent and natural learning experience. Students interact with the system as they would with paper by writing freely on a digital canvas while the system works continuously in the background to interpret handwriting, verify steps symbolically, monitor cognitive state, and intervene supportively when the student is struggling.

The system's IUI design is centred on three principles. At first, the naturalness of input: by accepting free-form handwriting and automatically converting it to LateX via a Vision LLM pipeline, ITIM removes the formal syntax barrier that prevents many students from using automated tutoring tools. Next, adaptive scaffolding: the ACT-R-inspired cognitive model continuously estimates each student's mastery and cognitive load, ensuring that interventions are triggered at the right moment, suppressed when the student is in flow, and calibrated to the minimum level of support that will restore productive engagement. Finally, Grounded conversational interaction: every LLM call is enriched with the student's current board content, exercise context, cognitive state, and interaction history, ensuring that the tutor responds to what the student is actually doing a design principle directly motivated by the findings of Ross et al. [6] in the programming assistance domain.

Together, these components realise all five of the project's stated goals: naturalness of interaction, adaptive feedback, misconception detection, exercise continuity, and multi-modal interaction. ITIM represents a step toward fully personalised, always-available mathematics tutoring that can serve students from middle school through university without requiring human tutors or formal syntax knowledge.

References

[1] Anderson, J. R. (1983). *The Architecture of Cognition*. Harvard University Press.

[2] Anderson, J. R., Boyle, C. F., and Reiser, B. J. (1985). Intelligent tutoring systems. *Science*, 228(4698), 456–462.

[3] Baker, R. S., D'Mello, S. K., Rodrigo, M. T., and Graesser, A. C. (2010). Better to be frustrated than bored: The incidence, persistence, and impact of learners' cognitive-affective states during interactions with three different computer-based learning environments. *International Journal of Human-Computer Studies*, 68(4), 223–241.

[4] Corbett, A. T. and Anderson, J. R. (1994). Knowledge tracing: Modelling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278.

[5] Heffernan, N. T. and Heffernan, C. L. (2014). The ASSISTments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 24(4), 470–497.

[6] Ross, S. I., Martinez, F., Houde, S., Muller, M., and Weisz, J. D. (2023). The Programmer's Assistant: Conversational interaction with a large language model for software development. In *28th International Conference on Intelligent User Interfaces (IUI '23)*, ACM, 491–514.

[7] Shute, V. J. (2008). Focus on formative feedback. *Review of Educational Research*, 78(1), 153–189.

A. Appendix: Complete File Reference

Table 5 lists all source files in the ITIM v2 repository with their line counts and roles. These are referenced throughout Section 3.

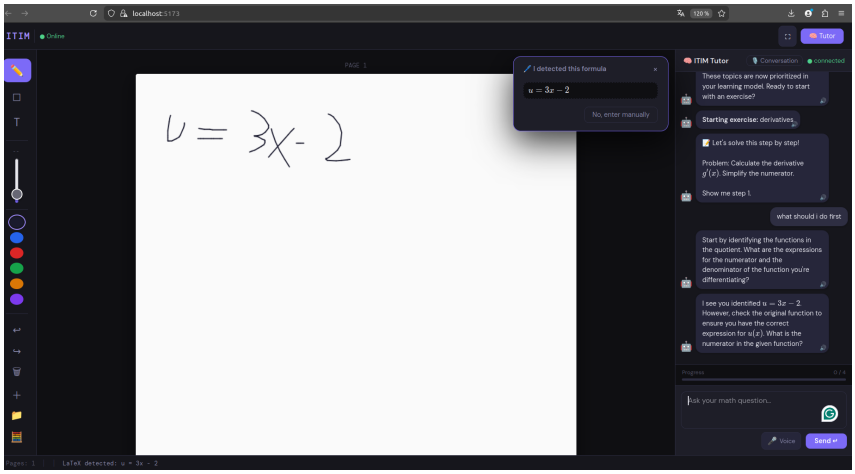


Figure 7: UI of ITIM

Table 5: All source files with line counts and descriptions.

File Description	Lines
Backend	
`app/main.py` FastAPI entry point, singletons, CORS, static file serving	81
`app/api_routes.py` REST endpoints: health, buggy rules, exercises, transcribe, TTS, OCR	230

Continued on next page

Table 5 (continued)

File Description	Lines
app/ws_handlers.py WebSocket message dispatcher and all real-time processing logic	1110
app/session.py SessionState per-connection container	53
app/intervention.py Proactive intervention background loop	32
core/llm_client.py Unified LLM interface: OpenAI + Groq + Mock fallback	277
core/connection_manager.py WebSocket connection registry	60
core/profile_manager.py Student profile persistence (SQLite)	120
domain/cognitive/act_r.py ACT-R cognitive engine	159
domain/cognitive/student_model.py Knowledge tracing and behavioural monitoring	150
domain/cognitive/pedagogical_agent.py Intervention decisions and scaffolding	200
domain/cognitive/dialogue_manager.py Conversation history management	80
domain/math/math_expert.py SymPy-based validation, buggy rule detection, step generation	520
domain/math/step_tracker.py Active exercise state, step evaluation, skip/give-up	244
domain/math/math_processor.py Vision API integration, LaTeX cleaning	180
domain/math/ast_utils.py AST conversion and comparison utilities	100
domain/exercises/exercise_generator.py LLM + SymPy exercise generation and verification	520
domain/exercises/exercise_library.py SQLite exercise persistence, search, deduplication	160
domain/exercises/document_processor.py Concept extraction from imported documents	140

Continued on next page

Table 5 (continued)

File Description	Lines
`domain/exercises/buggy_rule_learner.py` Incremental misconception learning	120
`input/stroke_buffer.py` Canvas stroke management	90
`input/stroke_analyzer.py` Frustration detection from eraser patterns	80
`tutor/tutor.py` Top-level pedagogical orchestrator	465
Frontend	
`src/App.jsx` Root component, WebSocket management, layout	431
`src/Page.jsx` Three-layer canvas, pointer events, text zones	757
`src/ChatPanel.jsx` Chat UI, conversation mode, VAD, TTS playback	280
`src/LatexConfirmToast.jsx` OCR result confirmation toast	142
`src/stores/useToolStore.jsx` Tool state (pen, eraser, highlighter, text)	60
`src/stores/usePageStore.jsx` Page and stroke state	100
`src/stores/useTutorStore.jsx` Exercise and tutor state	80
`src/stores/useUIStore.jsx` UI visibility and WebSocket URL	70
`src/hooks/useKeyboardShortcuts.js` Keyboard shortcut bindings	50
`src/hooks/useCanvasHistory.js` Undo/redo history management	60
`src/constants.js` Theme, colours, page dimensions	40
`index.html` KaTeX CDN, root element	24